

Phase 5

Networking Explained

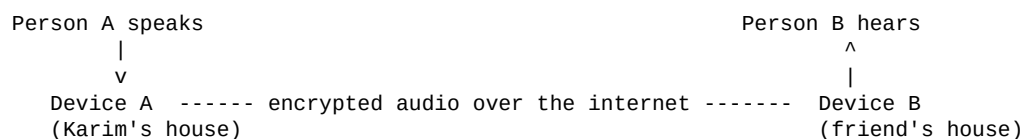
From scratch: TCP, NAT, the relay server, ngrok, and how it all fits with audio + encryption

This document explains **why we need a relay server**, **what TCP is**, **why ngrok**, and **how phone numbers work** in this project — all from first principles, no networking background assumed. It also documents what was built in Phase 5.1 / 5.2 / 5.3, and what comes next.

STATUS at end of Phase 5.3 (2026-04-28): Vodafone LTE modem successfully registers with relay.py through ngrok over 4G. Path verified end-to-end manually via Arduino Serial Monitor. Phase 5.4 (STM32 takes over from manual typing) is the next sub-task.

1. The Big Picture

The whole project goal is two devices that exchange encrypted voice over LTE. Each device has a microphone, a speaker, an STM32 (which captures, encrypts, decrypts, and plays back audio), and an LTE modem (which moves bytes between devices over the cellular network).



The challenge: **how do the two devices find each other and exchange bytes over the internet?**

2. The Networking Problem

On the internet, every machine has an **IP address** — a numeric label like 8.8.8.8 or 192.168.1.5. To send data to a machine, you specify its IP address.

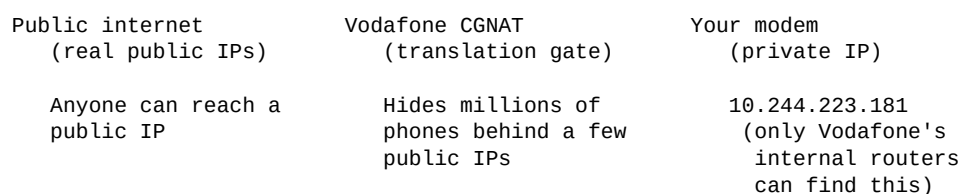
That sounds simple. But cellular networks have a problem.

2.1 — IP addresses are scarce

There are only ~4 billion IPv4 addresses in the world, but there are billions of phones, computers, IoT devices. There aren't enough public addresses to give each phone its own.

2.2 — Carriers use CGNAT

Carriers like Vodafone solve this with **Carrier-Grade NAT (CGNAT)**. Many subscribers share a small pool of public IPs. Each phone gets a **private IP** (like 10.244.223.181) that's only valid inside Vodafone's network.



2.3 — What this means in practice

Your modem can **call out** to public servers (Google, YouTube, etc.) — that works fine. But **nothing on the public internet can call IN to your modem**, because your modem has no public address.

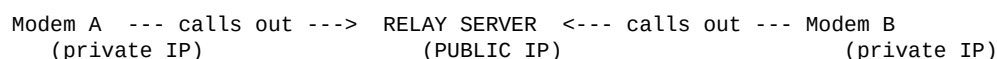
Your friend's modem has the exact same problem.

Therefore: the two modems cannot connect directly to each other. Both can go OUT, neither can be reached from the OUTSIDE.

This isn't a Vodafone bug — every consumer cellular carrier in the world does this. Your home Wi-Fi router does the same thing for your laptop and phone. It's how the internet is run today.

3. The Solution — A Relay Server in the Middle

Both modems can call OUT. So we put a server in the middle that has a public IP, and have both modems call it. The server then forwards messages between them.



|
v
Forwards bytes between
registered devices

The relay is just a small Python program (relay.py) that:

- Accepts TCP connections from devices.
- Remembers each device's phone number (used as a label, not as actual cellular calling).
- When device A says "CALL B", the relay finds B's connection and starts forwarding bytes.
- Doesn't read the bytes — just shovels them between A and B (preserves end-to-end encryption).

This is exactly how WhatsApp, Signal, FaceTime, Zoom, and every modern voice/video app works. The marketing makes them sound peer-to-peer but they all run massive server infrastructures because direct device-to-device just doesn't work on consumer networks.

4. What is TCP?

TCP (Transmission Control Protocol) is the simplest possible thing two computers can do over the internet: **open a reliable byte-stream channel between them.**

Think of it like a phone call:

- **Dial** — one side asks to connect to the other (TCP «handshake»).
- **Pickup** — the other side accepts.
- **Talk** — both sides can send bytes back and forth, in order, and nothing gets lost.
- **Hang up** — either side closes the connection.

That's it. TCP doesn't care what the bytes mean — it just delivers them reliably, in order, with no duplicates, no gaps. Your application (our relay, your modem firmware) decides what those bytes mean.

4.1 — TCP from the modem's side

The A7608 modem speaks TCP via these AT commands:

Command	TCP equivalent
AT+NETOPEN	Activate the IP/TCP stack inside the modem
AT+CIOOPEN=0,"TCP","host",port	Dial — open a connection to host:port (link 0)
AT+CIPSEND=0,N	Prepare to send N bytes on link 0
(then type N bytes)	Talk — send the bytes
+IPD N then bytes	Talk — receive N bytes from the other side
AT+CIPCLOSE=0	Hang up link 0

4.2 — Why TCP and not something else?

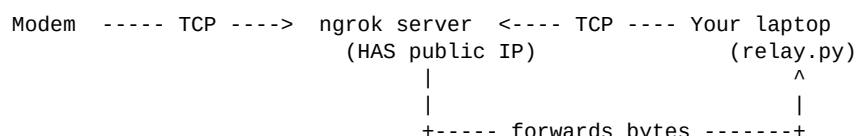
Two reasons we picked raw TCP for this project (instead of HTTP, MQTT, etc.):

- **It's the simplest layer.** TCP is built into every operating system and every modem firmware. No protocol library needed.
- **It gives us total control.** Our relay server can define any custom protocol on top — we used a simple line-based ASCII protocol with commands like REG, CALL, ANS, GO, HUP.
- **It's efficient for streaming.** For continuous audio bytes during a call, raw TCP has the lowest overhead.

5. Why ngrok?

There's still one missing piece: your relay (relay.py) runs on your laptop. **Your laptop is also behind your home router's NAT.** So your laptop doesn't have a public address either — the same problem as the modems.

ngrok is a service that gives you a temporary public address that forwards to your laptop:



So when you run `ngrok tcp 5555`, ngrok says: «OK, I have given you the public address `2.tcp.eu.ngrok.io:23515`. Anyone who connects there will be forwarded to your laptop's port 5555.» That's

where relay.py is listening.

The full path is:

```
Modem ----> Vodafone 4G ----> ngrok server ----> Your laptop ----> relay.py
                                   (public IP/port)      (port 5555)
```

ngrok is for development only. In a real product (or for the final demo if you want maximum reliability), you'd skip ngrok and run relay.py directly on a small public server (like a \$4/month DigitalOcean droplet). The modem would connect to the droplet's public IP directly.

6. How «Calling by Phone Number» Works

This was the question Karim asked early in the project. The answer: **phone numbers are just labels stored on the relay server**, NOT actual cellular voice calls.

The relay maintains a table:

Phone number	TCP socket
01000000001	Device A's TCP connection
01000000002	Device B's TCP connection

When both devices register with their phone numbers, the relay knows which socket goes to which device. From then on, any device can «call» another by phone number.

6.1 — The call flow

Step	Sender	Sends	What happens
1	Device A	REG 01000000001	A registers its number
2	Relay → A	OK	Confirmed
3	Device B	REG 01000000002	B registers its number
4	Relay → B	OK	Confirmed
5	Device A	CALL 01000000002	A wants to call B
6	Relay → A	RING	A: peer is ringing
7	Relay → B	INC 01000000001	B: A is calling you
8	Device B	ANS	B accepts
9	Relay → A	GO	Talk now
10	Relay → B	GO	Talk now
11	Both	audio bytes	Encrypted audio flows freely both ways
12	Either	HUP	Hang up
13	Relay → other	PEER_HUP	Peer hung up

6.2 — Why not use real cellular voice calls?

The A7608 modem can make real voice calls with ATD<number>;. But that conflicts with the project's encryption goal:

Real voice call:

```

Mic ---> LTE modem PCM bus ---> Cellular voice channel ---> Speaker
      |
      v
    STM32 NEVER sees the audio
      |
      v
    No encryption possible

```

This project (data + relay):

```

Mic ---> STM32 ADC ---> RSA + AES encrypt ---> LTE modem TCP send
      |
      v
    Encrypted bytes over LTE
      |
      v
    Other STM32 decrypts

```

|
v
Speaker

In the data path, every audio sample lives in the STM32's RAM at some point. That's what makes the encryption possible — the STM32 can transform the samples before they leave. Real voice calls bypass the STM32 entirely.

7. Phase 5 Sub-tasks — What's Done

Sub-task	What	Status
5.1	Run relay.py on laptop, listening on port 5555	✓ Done
5.2	Expose laptop port 5555 to public internet via ngrok TCP tunnel	✓ Done
5.3	Manually open TCP from LTE modem to ngrok address; send REG; relay confirms registration	✓ Done — verified end-to-end on 2026-04-28
5.4	STM32 takes over from manual Arduino Serial Monitor typing	NEXT
5.5	Hardware: wire LTE module to STM32 USART1 (with ESP32 reprogrammed as bridge)	After 5.4

7.1 — Working AT command sequence (verified)

```

AT
ATE0
AT+CPIN?          -> +CPIN: READY
AT+CSQ            -> +CSQ: 22,99    (signal >= 10 = good)
AT+CREG?          -> +CREG: 0,1    (registered home network)
AT+CGDCONT=1,"IP","internet.vodafone.net"
AT+CGACT=1,1
AT+NETOPEN        -> +NETOPEN: 0
AT+IPADDR         -> +IPADDR: 10.x.x.x
AT+CIPOPEN=0,"TCP","2.tcp.eu.ngrok.io",23515
                  -> +CIPOPEN: 0,0
AT+CIPSEND=0,17
                  -> > prompt
REG 010000000001<CR><LF>    (17 bytes total)
                  -> +CIPSEND: 0,17,17
                  -> +IPD 3
                  -> OK      (relay's reply travels back through ngrok)

```

And on the relay terminal:

```

+ conn ('127.0.0.1', xxxxx)
+ registered 010000000001

```

8. How Phase 5 Connects to Audio + Encryption

Phase 5 is just the network plumbing. Once it works, the rest of the project fits on top:

Phase 3 (DONE):

```
Mic -> MAX9814 -> STM32 ADC -> MCP4725 DAC -> LM386 -> Speaker
```

Phase 5 (in progress):

```
STM32 <----UART----> LTE modem <-- 4G --> ngrok --> relay.py
(transports any bytes both ways)
```

Phase 7 (LATER):

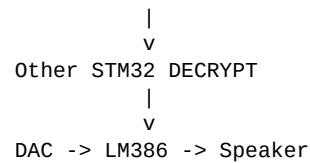
```
STM32 audio bytes ----encrypt----> bytes for Phase 5 transport
Phase 5 transport <----decrypt---- STM32 audio bytes
```

Putting it all together:

```

Mic -> ADC -> ENCRYPT -> UART -> LTE -> 4G -> ngrok -> relay.py
                                     |
                                     v
                                other relay/socket
                                     |
                                     v
                                Other LTE -> Other UART

```

Phase 5 only delivers a generic byte pipe. The audio+encryption logic (Phases 6 and 7) decide what bytes get pushed in and what comes out.

9. Phase 5.4 — Putting STM32 in Charge

So far, the human (Karim, typing in Arduino Serial Monitor) is the brain. Phase 5.4 replaces the human typing with STM32 firmware.

9.1 — The STM32 firmware approach

We **edit the existing MicDAC project** — do NOT start a new project. Reasons:

- MicDAC already has the audio path working (mic -> ADC -> DAC -> speaker, Phase 3).
- Phase 5.4 just **adds** LTE communication on top — doesn't replace anything.
- Starting a new project means redoing all the Phase 3 work, which would be wasteful.
- Audio code (TIM3 ISR, ADC DMA) keeps running in the background while LTE code drives the main loop.

9.2 — What gets added to MicDAC

Item	Where	Purpose
USART1 init (PA9 TX, PA10 RX)	CubeMX	Talk to LTE module (or to ESP32 bridge)
UART RX buffer + IDLE interrupt	main.c, USART1 IRQ handler	Receive AT replies
Function: send_at(cmd, expected, timeout)	main.c	Send an AT command and wait for response
LTE bring-up state machine	main.c USER CODE 4	Run AT, ATE0, CPIN?, CSQ, CGDCONT, CGACT, NETOPEN, CIPOPEN automatically
REG state machine	main.c USER CODE 4	Send REG , wait for OK, then idle
Status LED logic	main.c WHILE loop	PC13 reflects LTE state (booting, registered, in-call)

9.3 — Hardware swap

To put the STM32 in control of the LTE module, we need to either:

Option	How	Pros / Cons
(a) Reprogram ESP32 as STM32 bridge	Modify ESP32 sketch: bridge between STM32 UART and modem UART (instead of bridging USB)	Easier — keeps the working power-on sequence in ESP32. STM32 just sees a UART that talks to the modem.
(b) Bypass ESP32	Connect STM32 USART1 directly to the A7608 modem UART pins on the LilyGO board header	Cleaner — no ESP32 in the loop. Requires finding the A7608 UART pins on the specific LilyGO board.

Option (a) recommended — the existing ESP32 sketch already powers on the modem reliably; we just change where it gets bytes from (UART instead of USB).

10. Concrete — What Bytes Get Sent

During a full call, here is every byte that flows between Device A, the relay, and Device B. The bytes during «TALKING» are continuous; the rest are one-shot.

Phase	Sender	Bytes	Length
Setup	A → relay	REG 01000000001\r\n	17
Setup	relay → A	OK\r\n	3
Setup	B → relay	REG 01000000002\r\n	17
Setup	relay → B	OK\r\n	3
Call	A → relay	CALL 01000000002\r\n	18
Call	relay → A	RING\r\n	5
Call	relay → B	INC 01000000001\r\n	16
Answer	B → relay	ANS\r\n	5
Answer	relay → A	GO\r\n	3
Answer	relay → B	GO\r\n	3
Talking	A → relay → B	encrypted audio bytes (continuous)	variable
Talking	B → relay → A	encrypted audio bytes (continuous)	variable
Hangup	A or B → relay	HUP\r\n	5
Hangup	relay → other	PEER_HUP\r\n	9

During «Talking», encrypted audio frames are pushed continuously. The exact frame size depends on the encryption design (Phase 7) but typically 16–64 bytes per frame, ~125 frames per second at 8 kHz. Total bandwidth ~10 KB/sec one-way (2 KB/sec for the encrypted overhead, 8 KB/sec for the audio samples).

11. Summary — What's Now Possible

After Phase 5.3, this is the verified system:

- Two computer programs (relay.py + ngrok) running on the developer's laptop create a public TCP relay.
- Any device anywhere on the internet can connect to 2.tcp.eu.ngrok.io:NNNNN (or whatever ngrok assigns).
- Once connected, devices register a phone number with the relay.
- Devices identified by phone number can later request to call each other (Phase 6).
- The cellular path Vodafone 4G ↔ ngrok ↔ laptop ↔ relay.py is verified end-to-end with the LTE modem.
- All steps so far were done by typing AT commands manually in Arduino Serial Monitor; Phase 5.4 replaces the human with STM32 firmware.

Phase 5.4 next: STM32 firmware automates everything above so that on power-up, the device boots, brings up the LTE network, opens TCP to the relay, and registers its phone number — with no human intervention.

End of Phase 5 networking explanation.